Phase 1 静态 FJSP 数学建模

FJSP-AGV-RL Research Platform

2026 年 6 月 2 日

1 范围说明

本文档描述当前仓库在 Phase 1 中实现的静态柔性作业车间调度（Flexible Job Shop Scheduling Problem, FJSP）数学模型。当前阶段只关注静态 FJSP，不包含 AGV 运输、路径规划、动态事件、强化学习或图神经网络。

2 集合与下标

符号	含义
J	工件集合，下标为 j
O_j	工件 j 的有序工序集合，下标为 o
M	机器集合，下标为 m
A_{jo}	工序 (j,o) 的可选机器集合

对于每个工件 j，工序必须满足工艺路线顺序：(j,o) 必须先于 $(j,o+1)$ 加工。

3 参数

符号	含义
p_{jom}	工序 (j,o) 在机器 m 上的加工时间
H	保守的时间上界

当前 CP-SAT baseline 使用如下保守 horizon：

$$H = \sum_{j \in J} \sum_{o \in O_j} \max_{m \in A_{jo}} p_{jom}.$$

这个上界不一定紧，但对当前小规模 baseline 是安全的。

符号	含义
$x_{jom} \in \{0,1\}$	若工序 (j,o) 分配到机器 m，则取 1
$S_{jo} \geq 0$	工序 (j,o) 的开始时间
$C_{jo} \geq 0$	工序 (j,o) 的完成时间
$C_{\max} \geq 0$	最大完工时间，也就是 makespan
$y_{abm} \in \{0,1\}$	同一机器上两个工序 a,b 的先后顺序变量

4 决策变量

这里 a,b 表示类似 (j,o) 的工序编号。经典 MIP 写法可以显式使用 y 变量；当前 OR-Tools 实现则使用 optional interval 和 `NoOverlap` 表达机器容量约束。

5 目标函数

目标是最小化 makespan：

$$\min C_{\max}.$$

6 约束

6.1 机器选择约束

每道工序必须且只能选择一台可行机器：

$$\sum_{m \in A_{jo}} x_{jom} = 1, \qquad \forall j \in J,\ o \in O_j.$$

不在 A_{jo} 中的机器不是合法选择，代码中不会为这些机器生成候选分配。

6.2 加工时间连接约束

如果工序 (j,o) 选择机器 m，则完成时间必须等于开始时间加加工时间：

$$C_{jo} \geq S_{jo} + p_{jom} - H(1 - x_{jom}), \qquad \forall j,o,\ m \in A_{jo}.$$

6.3 工序顺序约束

同一工件内部必须满足工艺路线顺序：

$$C_{jo} \leq S_{j,o+1}, \qquad \forall j \in J,\ o, o+1 \in O_j.$$

6.4 机器容量约束

每台机器同一时刻最多只能加工一道工序。对任意两个不同工序 $a = (j,o)$ 和 $b = (k,q)$，若二者都可以选择机器 m，则需要满足：

$$S_a + p_{am} \leq S_b + H(1 - y_{abm}) + H(2 - x_{am} - x_{bm}),$$

$$S_b + p_{bm} \le S_a + H y_{abm} + H(2 - x_{am} - x_{bm}).$$

当两个工序都分配到机器 m 时，上述约束会强制二者不重叠。当前 CP-SAT 代码中，这部分由每台机器上的 `NoOverlap` 约束完成。

6.5 Makespan 定义

makespan 必须不小于每个工件最后一道工序的完成时间：

$$C_{\max} \ge C_{j,last(j)}, \qquad \forall j \in J.$$

7 Feasibility Checker 对应关系

代码中的 feasibility checker 会独立验证：

- 每道工序恰好出现一次；
- 工件编号和工序编号不越界；
- 每道工序选择的机器属于可选机器集合；
- 调度记录中的加工时间与实例数据一致；
- 同一工件内部满足工序顺序；
- 同一机器上的工序时间区间不重叠；
- 记录的 makespan 等于所有工序完成时间的最大值。

8 Dispatching Rule

严格 dispatching rule 共享候选元组：

$$(j, o, m, r_{jo}, s_{jom}, f_{jom}, p_{jom}),$$

其中 r_{jo} 是工件 ready time，s_{jom} 是在机器 m 上的最早可行开始时间，$f_{jom} = s_{jom} + p_{jom}$ 是候选完成时间。

规则	工序选择	机器选择
FIFO	最小 (r_{jo}, j, o)	最早完成机器
SPT	最小 p_{jom} 的 ready pair	与 pair 同时确定
EFT	最小 f_{jom} 的 ready pair	与 pair 同时确定
MWKR	剩余路线工作量最大	最早完成机器
MOPNR	剩余工序数最大	最早完成机器
Random	随机 ready pair	与 pair 同时确定

MWKR 的剩余路线工作量定义为：

$$W_{jo} = \sum_{q=o}^{last(j)} \min_{m \in A_{jq}} p_{jqm}.$$

MOPNR 使用剩余工序数作为 ready job 的优先级：

$$N_{jo} = last(j) - o + 1.$$

9 实验结果记录格式

Phase 1 实验脚本会在 `experiments/results/` 下输出标准 CSV。每一行表示一个算法在一个实例上的一次运行：

字段	含义
`experiment_id`	实验编号
`instance_name`	实例名称
`algorithm`	算法或求解器名称
`makespan`	目标值；若未返回调度则为空
`feasible`	feasibility checker 的结果，PASS 或 FAIL
`runtime_s`	运行时间，单位为秒
`note`	求解器状态、错误信息或验证说明
`objective_bound`	CP-SAT 给出的目标下界，若可用
`gap`	相对 gap，定义见下节，若可用
`solver_wall_time`	CP-SAT 内部 wall time，若可用

CSV 记录层只负责保存实验事实，不改变算法行为，也不改变可行性定义。Phase 1 实验脚本使用共用 runner helper，由 helper 统一负责计时、调用求解器、运行独立 feasibility checker、收集失败信息，并将可选的 solver metadata 写入 CSV 记录。

9.1 Benchmark 批量实验入口

批量实验入口会按照稳定路径顺序递归发现 `*.fjs` 文件。对于每个成功加载的静态 FJSP 实例，入口会运行所选的 Phase 1 启发式算法，并默认运行 CP-SAT 参考求解器。CP-SAT 的单实例 time limit 可以配置，从而限制较大 benchmark 集合上的计算时间。若文件格式错误，入口会保留一条 `LOAD` 算法、`FAIL` 状态的记录，避免损坏文件从实验报告中静默消失。

9.2 已跟踪的 FJSPLib 子集

仓库会跟踪一个较小的真实数据子集。文件来自 ScheduleOpt 官方 FJSPLib 仓库，并按照其 CC-BY-SA-4.0 许可证保留来源说明：`https://scheduleopt.github.io/benchmarks/fjsplib`。该子集覆盖四个来源系列和逐步增大的实例规模。

实例	工件数	机器数	FJSPLib BKS	本地 CP-SAT
`fattahi1`	2	2	66	66
`fattahi2`	2	2	107	107
`fattahi3`	3	2	221	221
`kacem1`	4	5	11	11
`kacem2`	10	7	11	11
Hurink edata `ft06`	6	6	55	55
Brandimarte `mk01`	10	6	40	40
Brandimarte `mk02`	10	6	26	28
Brandimarte `mk03`	15	8	204	206
Brandimarte `mk04`	15	8	60	61

上表中的本地 CP-SAT 结果均通过独立 feasibility checker。FJSPLib BKS 列来自上游参考数据，不是本仓库自行推断的值。对于 `mk02`、`mk03` 和 `mk04`，CP-SAT 在 10 秒时限内报告 `FEASIBLE`。因此批量实验只记录限时可行参考调度，不声称本地运行证明了最优性。

官方 benchmark 网页将 `kacem1` 标为 4×6，但官方旧格式文件以 `4 5 5` 开头。本仓库按照可加载文件内容记录为 4×5，并保留上游 BKS 11。

10 CP-SAT 诊断信息

对于 CP-SAT 结果，代码会在通用的 `ScheduleResult` 对象中保存三个可选诊断字段：

$$LB = \text{BestObjectiveBound}, \qquad gap = \frac{C_{\max} - LB}{\max(1, C_{\max})}.$$

此外还会记录求解器内部的 wall time。对于 toy instance 上已证明最优的解，下界等于 makespan，因此 gap 为 0。启发式算法没有这些求解器诊断信息，对应 CSV 字段保持为空。

10.1 CP-SAT 时限 sweep

tracked `mk03` sweep 将 CP-SAT worker 数固定为 1，只改变 time limit。固定 worker 数可以减少并行搜索噪声，使对比更容易解释。

时限（秒）	Makespan	状态	Bound	Gap
1	417	`FEASIBLE`	20	0.952038
3	285	`FEASIBLE`	20	0.929825
10	220	`FEASIBLE`	20	0.909091

在本次运行中，更长时限改善了可行调度，但没有任何一行证明最优性。默认 benchmark 批量实验使用求解器默认 worker 配置，并单独记录。worker 数属于实验配置的一部分，比较时不能隐藏。

10.2 CP-SAT worker 数量 sweep

tracked mk03 worker sweep 将时限固定为 10 秒，只改变 CP-SAT worker 配置。auto 表示保留求解器的默认 worker 选择。

Workers	Makespan	状态	Bound	Gap
1	220	FEASIBLE	20	0.909091
2	207	FEASIBLE	20	0.903382
4	207	FEASIBLE	20	0.903382
auto	209	FEASIBLE	75	0.641148

每一行均通过独立 feasibility checker。该表只是一次限时运行的观测结果，不是稳定的 worker 性能排名：并行 CP-SAT 搜索可能在不同运行之间产生变化。本次观测中，自动配置给出了更强的 bound，而 2 和 4 workers 给出了略好的 incumbent。

10.3 CP-SAT worker 重复观测

重复实验保持相同的 mk03 实例和 10 秒时限，对每种 worker 配置运行 3 次。实验同时保存每一条经过 feasibility checker 的原始记录和单独的聚合 CSV。表中的标准差是观测 makespan 的总体标准差。

Workers	通过数	最佳值	平均值	标准差	最佳 bound	最佳 gap
1	3/3	220	220.000000	0.000000	20	0.909091
2	3/3	204	204.000000	0.000000	20	0.901961
4	3/3	204	204.000000	0.000000	21	0.897059
auto	3/3	204	204.000000	0.000000	88	0.568627

全部 12 个调度均通过独立 feasibility checker。在这个小样本中，2、4 和自动 worker 都稳定达到上游 BKS 204。但是 CP-SAT 仍然报告 FEASIBLE，所以这些结果仍然只是限时可行参考解，不是本地已证明最优解。

10.4 CP-SAT benchmark 重复矩阵

受控矩阵递归扫描 tracked 静态 FJSPLib 子集。每个实例在 3 秒时限下使用单 worker 运行 2 次，再使用 auto worker 配置运行 2 次。逐次原始记录和按实例聚合的统计记录分别保存。

全部 40 个调度均通过独立 feasibility checker。除 mk02 外，较小的 tracked 实例在时限内证明了最优性。对于 mk02、mk03 和 mk04，auto workers 改善了本次观测中的 incumbent 和 bound，但两种配置都没有在 3 秒内证明最优性。

10.5 启发式 benchmark 矩阵

启发式矩阵在每个 tracked FJSPLib 实例上运行一次确定性 Phase 1 规则。两类随机算法使用固定 seed 42、123 和 999。因此，表中的范围是可复现观测，而不是没有控制的随机样本。

实例	Workers	通过数	最佳值	平均值	状态	Bound
mk01	1	2/2	40	40.000000	OPTIMAL	40
mk01	auto	2/2	40	40.000000	OPTIMAL	40
mk02	1	2/2	31	31.000000	FEASIBLE	3
mk02	auto	2/2	30	30.000000	FEASIBLE	20
mk03	1	2/2	344	348.000000	FEASIBLE	20
mk03	auto	2/2	230	230.500000	FEASIBLE	60
mk04	1	2/2	68	68.000000	FEASIBLE	14
mk04	auto	2/2	62	63.000000	FEASIBLE	41
fattahi1	1/auto	4/4	66	66.000000	OPTIMAL	66
fattahi2	1/auto	4/4	107	107.000000	OPTIMAL	107
fattahi3	1/auto	4/4	221	221.000000	OPTIMAL	221
ft06	1/auto	4/4	55	55.000000	OPTIMAL	55
kacem1	1/auto	4/4	11	11.000000	OPTIMAL	11
kacem2	1/auto	4/4	11	11.000000	OPTIMAL	11

全部 140 个调度均通过独立 feasibility checker。没有一个规则在所有实例上占优：MWKR 在 mk01、mk02、fattahi1 和 kacem2 上最好；FIFO 在 mk04、ft06 和 kacem1 上最好，在 mk03 和 fattahi2 上与 MWKR 并列，并在 fattahi3 上与 MOPNR 并列。ready-operation 随机 dispatching 在当前 tracked 子集上持续优于简单 job-order 随机 baseline，但仍然受 seed 影响。

实例	最佳确定性规则	Random 范围	DispatchRandom 范围
`mk01`	`DispatchMWKR` (44)	148–165	92–125
`mk02`	`DispatchMWKR` (30)	134–169	86–113
`mk03`	`DispatchFIFO/MWKR` (204)	1086–1206	488–563
`mk04`	`DispatchFIFO` (77)	279–292	143–177
`fattahi1`	`DispatchMWKR` (66)	86–135	86–135
`fattahi2`	`DispatchFIFO/MWKR` (107)	107–192	107–149
`fattahi3`	`DispatchFIFO/MOPNR` (221)	298–442	221–361
`ft06`	`DispatchFIFO` (55)	146–152	71–112
`kacem1`	`DispatchFIFO` (11)	27–82	26–44
`kacem2`	`DispatchMWKR` (12)	62–102	50–88

10.6 基于 BKS 的启发式质量指标

质量报告从 `instances/fjsplib_bks.csv` 读取结构化上游参考值。对于启发式算法族 h 和实例 i，计算

$$gap_{\mathrm{BKS}}(h,i) = \frac{C_{\max}(h,i) - \mathrm{BKS}_i}{\mathrm{BKS}_i}.$$

上游 BKS 用于比较，不等于本地求解器已经证明最优性。

算法族	平均最佳 gap	最大最佳 gap
FIFO	3.856359	14.818182
SPT	1.713594	3.366667
EarliestFinishTime	0.498406	1.384615
DispatchFIFO	0.120140	0.303030
DispatchSPT	0.978694	2.066667
DispatchEFT	0.500318	1.384615
DispatchMWKR	0.099709	0.316667
DispatchMOPNR	0.189633	0.466667
Random	2.322428	4.636364
DispatchRandom	1.188621	3.545455

在当前十实例子集上，DispatchMWKR 的平均最佳 gap 最小。当前样本仍然较小，应优先扩展 benchmark 覆盖，再决定是否增加新规则。

10.7 Tracked benchmark 元数据审计

在扩展 benchmark 子集之前，元数据审计会递归发现 tracked FJSPLib 文件，将其与 `instances/fjsplib_bks.csv` 逐项匹配，并加载每个发现的文件。只有 tracked 文件存在、上游参考记录存在、loader 接受该实例时，一行记录才会通过。当前十行均通过。这个门禁可以避免遗漏文件或过期元数据悄悄进入质量比较。